#1 CUDA Matrix Transposition

```
!nvcc transpose.cu -o transpose
!./transpose
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compute/sm/lto>_75' will be removed in a
================================================
 Experiment: 2048 x 1024 | Block Size: [16, 16]
================================================
 CPU Time              :    59.1300 ms
 GPU Naive (Kernel)    :     0.3419 ms  (Correct: YES)
 GPU Naive (TOTAL)     :     8.6190 ms
 GPU Optimized (Kernel):     0.1292 ms  (Correct: YES)
 GPU Optimized (TOTAL) :     5.3030 ms
 Kernel Speedup        : x2.65
 TOTAL Operation Speedup: x1.63

================================================
 Experiment: 4096 x 2048 | Block Size: [16, 16]
================================================
 CPU Time              :   232.6910 ms
 GPU Naive (Kernel)    :     0.5109 ms  (Correct: YES)
 GPU Naive (TOTAL)     :    30.8310 ms
 GPU Optimized (Kernel):     0.5117 ms  (Correct: YES)
 GPU Optimized (TOTAL) :    21.0480 ms
 Kernel Speedup        : x1.00
 TOTAL Operation Speedup: x1.46


================================================
 Experiment: 4096 x 2048 | Block Size: [32, 32]
================================================
 CPU Time              :   216.6500 ms
 GPU Naive (Kernel)    :     0.9717 ms  (Correct: YES)
 GPU Naive (TOTAL)     :    31.2020 ms
 GPU Optimized (Kernel):     0.9751 ms  (Correct: YES)
 GPU Optimized (TOTAL) :    22.6780 ms
 Kernel Speedup        : x1.00
 TOTAL Operation Speedup: x1.38

================================================
 Experiment: 4096 x 2048 | Block Size: [32, 8]
================================================
 CPU Time              :   227.9460 ms
 GPU Naive (Kernel)    :     0.9880 ms  (Correct: YES)
 GPU Naive (TOTAL)     :    30.7670 ms
 GPU Optimized (Kernel):     0.9848 ms  (Correct: YES)
 GPU Optimized (TOTAL) :    21.6600 ms
 Kernel Speedup        : x1.00
 TOTAL Operation Speedup: x1.42
```

1. Introduction

Matrix transposition (B = A^T) is a fundamental operation in linear algebra. In CUDA programming, this task serves as a critical benchmark for analyzing memory bandwidth, coalesced access patterns, and the efficiency of different memory management models (Standard Device Memory vs. Unified Memory).

2. Implementation Approaches

CPU Sequential: A baseline implementation using nested loops to provide a reference for functional correctness and sequential performance.

GPU Naive: A standard CUDA implementation using cudaMalloc and synchronous cudaMemcpy. Each thread maps to an output element, resulting in uncoalesced memory writes.

GPU Optimized: An implementation leveraging **Unified Memory** (cudaMallocManaged) paired with **Asynchronous Prefetching** (cudaMemPrefetchAsync). This shifts the data migration overhead to the background, overlapping it with kernel launch preparations.

3. Analysis and Observations

Kernel Parity: The kernel execution times are identical. This confirms that both implementations share the same underlying memory access limitations (uncoalesced writes) that bottleneck the GPU's memory controller.

Systemic Efficiency: The Optimized implementation provides a **1.46x total speedup**. While the raw math throughput is identical, the Optimized version eliminates the blocking latency of synchronous cudaMemcpy operations by utilizing background asynchronous prefetching.

Topology Impact: Testing different block configurations revealed that **16 ×16** blocks provide the optimal balance for GPU occupancy. Larger blocks (32 ×32) increase scheduling pressure, while asymmetric blocks (32 ×8) degrade write alignment, further hindering memory performance.

4. Potential Further Optimizations

The current implementations are limited by global memory access patterns. To achieve higher performance, the following optimizations are proposed:

Shared Memory Tiling: Loading data into on-chip **Shared Memory** would allow threads to perform the transposition in a high-speed cache. This enables subsequent writes to Global Memory to be performed in a fully coalesced, row-wise manner.

Bank Conflict Avoidance: Padding the shared memory arrays (e.g., tile[TILE_DIM][TILE_DIM + 1]) would prevent memory bank conflicts during column-wise reads, ensuring the hardware can sustain peak throughput.

Vectorized Loads/Stores: Implementing float4 access patterns could reduce the instruction count and fully saturate the memory bus width.